

27 — Search Analytics: Logging, Aggregation, and Quality Feedback

This doc covers the search analytics system — how every search query is logged, how those logs are aggregated into actionable insights, and how those insights feed back into improving search quality.

What it is

Every time a user (or a chatbot integration) hits a search endpoint, the system records: what was searched, which endpoint was called, what language was detected, how many results came back, and how long it took. These records are stored in a `search_analytics` table in PostgreSQL.

An admin dashboard then aggregates this raw data into three views: 1. **Top queries** — the most frequently searched terms, with average response times. 2. **Zero-result queries** — searches that returned nothing, ranked by how often they occur. 3. **Recent searches** — a live feed of the last 20 queries with full details.

This is not a complex analytics platform — it's a focused feedback loop that answers one question: "Is the search working well for the things people are actually searching for?"

Real-life analogy

Imagine you run a library reference desk. Every time someone asks a question, you write it in a logbook: the question, when they asked, whether you found an answer, and how long it took you.

At the end of each week, you review the logbook and notice patterns:

- "3D printing" was asked 47 times — that's popular, you should make sure those books are prominently shelved.
- "quantum biology" was asked 12 times and you found nothing every time — that's a gap in your collection. You need to order those books.
- The average answer time for "Hungarian cooking" searches was 45 seconds — suspiciously slow. Maybe the Hungarian section needs reorganization.

The logbook is the `search_analytics` table. The weekly review is the admin analytics dashboard. The action items (reorganize a section, order new books) are what the admin does based on the analytics — tweaking system prompts, adding protected terms, adjusting provider settings.

Why we used it here

A search system without analytics is flying blind. You can have the most sophisticated hybrid vector + BM25 + RRF pipeline in the world, but if you don't know what people are actually searching for, you can't tell if it's working.

Three concrete scenarios where analytics drive improvement in this project:

1. **Zero-result queries reveal coverage gaps.** If "elektromos kerekpar" (electric bicycle) returns zero results 30 times, that's a signal: either the data doesn't include electric bicycle companies (data gap), or the narrative/embedding doesn't connect that Hungarian phrase to the relevant categories (search quality gap).

Without analytics, that failure is invisible — users just get empty results and move on.

2. **Response time outliers reveal performance issues.** If most queries return in 200ms but "industrial automation" consistently takes 2 seconds, something is off — maybe the BM25 path is doing a full table scan for that term, or the embedding dimension is too high for the current hardware.
3. **Top queries inform prompt tuning.** If the top 5 search terms are all in Hungarian but the system is returning English results, the language detection (doc 04) or cross-language fallback (doc 16) might need adjustment. You'd see this pattern in the analytics dashboard before users even report it.

Code walkthrough

Layer 1: Logging every search

From `search_api/services/search.py`:

Table creation (lazy, once per server lifetime):

```
_ANALYTICS_TABLE_ENSURED = False

def _ensure_analytics_table(conn) -> None:
    global _ANALYTICS_TABLE_ENSURED
    if _ANALYTICS_TABLE_ENSURED:
        return
    try:
        with conn.cursor() as cur:
            cur.execute("""
                CREATE TABLE IF NOT EXISTS search_analytics (
                    id SERIAL PRIMARY KEY,
                    query TEXT NOT NULL,
                    endpoint TEXT NOT NULL,
                    language TEXT,
                    result_count INT,
                    response_time_ms FLOAT,
                    created_at TIMESTAMPTZ DEFAULT NOW()
                )
            """)
    except:
```

```

        )
        """)
        conn.commit()
        _ANALYTICS_TABLE_ENSURED = True
    except Exception:
        pass

```

The module-level `_ANALYTICS_TABLE_ENSURED` flag means the `CREATE TABLE IF NOT EXISTS` runs at most once per API server process. After the first successful creation, all subsequent calls skip the check entirely. This avoids running a DDL statement on every single search request.

The `SERIAL PRIMARY KEY` auto-generates incrementing IDs.

`created_at` defaults to the current timestamp via `DEFAULT NOW()`.

The table schema stores exactly the five dimensions we care about: what (query), where (endpoint), what language, how many results, how long it took.

The logging function:

```

def log_search_analytics(
    query: str, endpoint: str, language: str,
    result_count: int, response_time_ms: float
) -> None:
    try:
        conn = get_connection()
        try:
            _ensure_analytics_table(conn)
            with conn.cursor() as cur:
                cur.execute(
                    "INSERT INTO search_analytics "
                    "(query, endpoint, language, result_count, response_time_ms) "
                    "VALUES (%s, %s, %s, %s, %s)",
                    (query, endpoint, language, result_count, response_time_ms),
                )
            conn.commit()
        finally:
            conn.close()
    except Exception:
        pass

```

Two critical design choices here:

1. **Own connection.** The function opens and closes its own database connection rather than sharing the search query's connection. This isolates analytics from the search path — if the analytics insert fails or is slow, it doesn't affect the search response that's already been sent to the user.
2. **Silent failure.** The outer `try/except: pass` means analytics logging never raises an exception to the caller. If the database is temporarily unreachable, the search still works — you lose one analytics record, not the search result. Analytics is a side effect, not part of the critical path.

Where it's called (from each router):

```
# In category.py:
log_search_analytics(req.query, "category", raw["detected_language"],
                    len(results), raw["response_time_ms"])

# In companies.py (match):
log_search_analytics(req.query, "companies", detected_lang,
                    len(results_out), elapsed)

# In companies.py (single-company):
log_search_analytics(req.query, "single_company", detected_lang,
                    len(results_out), elapsed)
```

Every search endpoint calls `log_search_analytics` after computing results. The endpoint name string ("category", "companies", "single_company") lets the admin see which endpoint is being used most and which has the most zero-result queries.

Layer 2: The aggregation endpoint

From `search_api/routers/admin_config.py`:

```
@etl_router.get("/admin/analytics")
def get_analytics():
    conn = get_connection()
```

```

try:
    with conn.cursor() as cur:
        # Safety check: does the table exist?
        cur.execute("""
            SELECT EXISTS (
                SELECT FROM information_schema.tables
                WHERE table_name = 'search_analytics'
            )
        """)
        if not cur.fetchone()[0]:
            return {"total_queries": 0, "zero_result_queries": [],
                    "top_queries": [], "recent": []}

        # Total query count
        cur.execute("SELECT COUNT(*) FROM search_analytics")
        total = cur.fetchone()[0]

        # Top zero-result queries
        cur.execute("""
            SELECT query, COUNT(*) as cnt
            FROM search_analytics WHERE result_count = 0
            GROUP BY query ORDER BY cnt DESC LIMIT 10
        """)
        zero_results = [{"query": r[0], "count": r[1]}
                        for r in cur.fetchall()]

        # Top queries with average response time
        cur.execute("""
            SELECT query, COUNT(*) as cnt,
                ROUND(AVG(response_time_ms)::numeric, 1) as avg_ms
            FROM search_analytics
            GROUP BY query ORDER BY cnt DESC LIMIT 10
        """)
        top_queries = [{"query": r[0], "count": r[1], "avg_ms": float(r[2])}
                       for r in cur.fetchall()]

        # Recent 20 queries
        cur.execute("""
            SELECT query, endpoint, language, result_count,
                response_time_ms, created_at
            FROM search_analytics ORDER BY created_at DESC LIMIT 20
        """)
        recent = [...]

finally:

```

```
conn.close()

return {"total_queries": total, "zero_result_queries": zero_results,
        "top_queries": top_queries, "recent": recent}
```

Four SQL queries in one endpoint call:

1. **Existence check** — if the table doesn't exist (no searches have happened yet), return empty data immediately. This avoids a SQL error on a fresh deployment.
2. **Total count** — `COUNT(*)` across all records.
3. **Zero-result aggregation** — `GROUP BY` query `WHERE result_count = 0`, sorted by frequency. This answers "what are people searching for that we can't answer?"
4. **Top queries** — `GROUP BY` query with `COUNT` and `AVG(response_time_ms)`. This answers "what are people searching for most, and how fast are we answering?"

The `LIMIT 10` on aggregations and `LIMIT 20` on recent queries keeps the response small and fast. At this project's scale (hundreds to low thousands of logged queries), these queries are instant. At larger scale, you'd add indexes on `(query)` and `(created_at)`, but that's premature here.

Layer 3: The React dashboard

From `admin-ui/src/pages/Analytics.jsx`:

Data fetching:

```
useEffect(() => {
  fetch(`${API_BASE}/admin/analytics`)
    .then(r => r.json())
    .then(setData)
    .catch(() => {});
}, []);
```

Unlike the ETL progress page (which polls every 5 seconds), the analytics page fetches once on mount. Analytics data doesn't change in

real time the way ETL progress does — it's a historical summary that updates meaningfully over hours or days, not seconds.

Three data views rendered conditionally:

1. **Stat cards** at the top: total queries, number of unique popular terms, number of zero-result queries. The zero-result count turns red (`var(--danger)`) when non-zero — it's a visual alert that there are searches the system can't answer.
2. **Top Queries table**: query text, number of times searched, and average response time in milliseconds. This tells the admin which searches are most important to optimize.
3. **Zero-Result Queries table**: query text and occurrence count. These are the highest-priority improvement targets — people are searching for something and getting nothing.
4. **Recent Searches table**: a live feed showing the last 20 queries with endpoint, language, result count, and response time. This is useful for real-time debugging: "I just ran a search and it felt slow — let me check the analytics page to see the actual response time."

Color coding:

```
<span className={`badge ${r.result_count === 0 ? "badge-danger" : "badge-neutral"}  
  {r.result_count}  
</span>
```

Zero-result entries get a red badge; non-zero results get a neutral one. This pattern repeats throughout the dashboard — the design draws the admin's eye to problems (red zeros) rather than requiring them to scan a table of numbers looking for issues.

The feedback loop

Analytics are not just for observation — they close a loop:

1. Admin sees "elektromos kerekpar" has 30 zero-result searches.
2. Admin checks the data: are there electric bicycle companies in the source data? Yes.
3. The issue is that the narrative doesn't include "elektromos kerekpar" as a searchable term.
4. Admin goes to the Prompt Editor (doc to be written) and adjusts the enrichment system prompt to generate more Hungarian keyword synonyms.
5. Admin re-runs the ETL with `--force-enrichment`.
6. Next time someone searches "elektromos kerekpar," it returns results.
7. The zero-result count for that term stops growing.

Without analytics, step 1 never happens — the problem is invisible.

How to explain this in an interview

"We built a search analytics pipeline with three layers: every search endpoint inserts a row into a `search_analytics` table recording the query, endpoint, language, result count, and response time — using a separate connection and silent failure so it never affects search performance. An aggregation endpoint runs SQL GROUP BY queries to surface the top searched terms, zero-result queries, and recent activity. A React dashboard displays these as stat cards and sortable tables, with red highlighting on zero-result entries. The key value isn't just monitoring — it's a feedback loop: zero-result queries reveal coverage gaps that the admin can fix by adjusting system prompts or adding data, and they can verify the fix worked by watching the zero-result count drop. The whole system uses nothing beyond PostgreSQL and HTTP — no external analytics service, no event streaming, just SQL aggregation over a single table."